# Informe tècnic complet - Lab 1 VC/PSIV

## 1) Objectiu del projecte i què resol exactament el codi

Aquest projecte implementa un sistema complet de **detecció de vehicles en una seqüència de trànsit** amb l'enfoc de **background subtraction**, seguint les tasques del laboratori.

El flux funcional del codi Python és:

1. Carregar llistes de fitxers i imatges (`splits/*.txt`) en escala de grisos.
2. Construir un model de fons amb **mitjana** (`mu`) i **desviació estàndard** (`sigma`) sobre `train`.
3. Fer segmentació simple (`|I - mu| > thr`).
4. Fer segmentació gaussiana (`|I - mu| > alpha * sigma + beta`) amb 3 proves de paràmetres.
5. Aplicar postprocessat morfològic (`opening + closing`).
6. Generar vídeos de visualització amb overlay.
7. Avaluar quantitativament contra `groundtruth` (`accuracy`, `precision`, `recall`, `F1`).

La idea central és separar **fons** i **objectes en moviment**, i després validar numèricament com de bona és la segmentació.

## 2) Arquitectura i filosofia de disseny

El codi està fet en mòduls funcionals simples (funcions independents) i un `main()` que orquestra el pipeline. Aquesta decisió facilita:

1. Testejar cada peça per separat.
2. Reutilitzar funcions per diferents experiments.
3. Explicar la lògica en defensa oral sense perdre's en un script monolític.

S'ha prioritzat:

1. Robustesa d'entrada/sortida (errors clars si falten fitxers o si una imatge no es pot llegir).
2. Càlcul vectoritzat (sense bucles per píxel) per eficiència.
3. Compatibilitat i simplicitat d'entorn (només `cv2` + `numpy`).

## 3) Explicació detallada, funció per funció

### 3.1 Imports i dependències

Referència: main.py:1, main.py:4, main.py:5

1. `pathlib.Path` per rutes portables.
2. `cv2` per lectura/escriptura d'imatges, morfologia i vídeo.
3. `numpy` per càlcul numèric i operacions vectoritzades.

Decisió important: s'han eliminat llibreries de backup (`PIL`, `imageio`) per simplificar el sistema i evitar comportaments divergents entre entorns.

### 3.2 Lectura de llistes de fitxers

Referència: main.py:8

`read_list_file(path)`:

1. Llegeix un fitxer `.txt` amb una ruta per línia.
2. Fa `strip` de cada línia.
3. Elimina línies buides.
4. Retorna una llista de `Path`.

Per què és bona pràctica:

1. Separa definició de dades del codi.
2. Permet reproduïbilitat (sempre mateixes imatges i ordre).

### 3.3 Lectura d'imatges en grisos

Referència: main.py:13

`read_gray_image(path)`:

1. Usa `cv2.imread(..., cv2.IMREAD_GRAYSCALE)`.
2. Si falla, llença `RuntimeError` amb context.

Punt de validació oral:

1. El laboratori demana treball en grisos per simplificar model i reduir dimensions.
2. Treballar en un canal accelera càlcul i evita decidir com combinar canals RGB.

### 3.4 Guardat d'imatges

Referència: main.py:22

`save_gray_image(path, imatge)`:

1. Crea carpeta si no existeix.
2. Fa `clip` al rang `[0,255]`.
3. Converteix a `uint8`.
4. Desa amb `cv2.imwrite`.

Per què cal `clip + uint8`:

1. `mu` i `sigma` són `float32`.
2. Les imatges de sortida han de ser 8 bits per visualització estàndard.

### 3.5 Càrrega d'un stack d'imatges (tensor)

Referència: main.py:30

`load_stack(paths, nom)` construeix un tensor `N x H x W`:

1. Verifica que la llista no sigui buida.
2. Comprova existència de cada fitxer.
3. Llegeix cada imatge en grisos.
4. Verifica consistència de dimensions.
5. Apila amb `np.stack(axis=0)`.

Això evita bugs silenciosos, per exemple:

1. Imatge trencada amb mida diferent.
2. Path mal escrit.

### 3.6 Model de fons: `mu` i `sigma`

Referència: main.py:59

`calcula_model_fons(train_input)`:

1. Converteix `train_input` a `float32`.
2. `mu = mean(train, axis=0)`.
3. `sigma = std(train, axis=0)`.

Raó crítica de `float32`:

1. Amb `uint8`, restes com `10 - 200` fan wrapping (resultat incorrecte).
2. En flotant, la diferència és matemàticament correcta.

Significat físic:

1. `mu(i,j)` és la intensitat mitjana esperada del fons al píxel `(i,j)`.
2. `sigma(i,j)` mesura variabilitat local del píxel al temps.

### 3.7 Normalització per visualitzar `sigma`

Referència: main.py:66

`normalitza_per_visualitzar`:

1. Troba min i max.
2. Escala linealment a `[0,255]`.
3. Si imatge constant (`max <= min`), retorna zeros.

És una operació de visualització, no canvia el model matemàtic usat a segmentació.

### 3.8 Segmentació simple

Referència: main.py:75

`segmenta_simple(test_input, mu, thr)`:

1. `diferencia = abs(I - mu)` en `float32`.
2. `mascara = diferencia > thr`.
3. Converteix booleà a `uint8` 0/255.

Interpretació:

1. Si el píxel difereix poc del fons, és background.
2. Si difereix molt, és foreground.

Limitació:

1. Un `thr` global no s'adapta a zones amb soroll diferent.

### 3.9 Segmentació gaussiana (model adaptatiu)

Referència: main.py:82

`segmenta_model_gaussia(test_input, mu, sigma, alpha, beta)`:

1. Calcula `abs(I - mu)`.
2. Defineix llindar per píxel `alpha * sigma + beta`.
3. Marca foreground si supera aquest llindar.

Interpretació de paràmetres:

1. `alpha` incrementa/reduiex pes de variabilitat local.
2. `beta` desplaça el llindar globalment.
3. Més `alpha/beta` implica sistema més conservador (menys deteccions, menys falsos positius, però més falsos negatius).

### 3.10 Guardat de màscares

Referència: main.py:92

`desa_mascares_binarias`:

1. Usa el nom original (`inXXXXXX`) per construir `fgXXXXXX.png`.
2. Manté alineació temporal amb frames de test.

Això és clau per traçabilitat i debug.

### 3.11 Morfologia

Referència: main.py:100

`aplica_morfologia(mascares, mida_open=3, mida_close=5)`:

1. `opening` elimina soroll petit (punts blancs espuris).
2. `closing` omple forats petits dins objectes.

Ordre triat:

1. Primer neteja soroll (`open`).
2. Després compacta objectes (`close`).

### 3.12 Vídeos de resultats

Referència: main.py:114

`crea_video_resultats`:

1. Crea `VideoWriter` amb `mp4v`.
2. Per frame, converteix gris a BGR.
3. Pinta foreground en vermell.
4. Escriu frame a vídeo.

La superposició en vermell facilita inspecció visual de qualitat.

### 3.13 Avaluació de mètriques

Referència: main.py:138

`avalua_mascares(mascares_pred, gt)`:

1. Tracta foreground predit com `pred > 0`.
2. Interpreta etiquetes CDnet:
3. `0` fons.
4. `50` ombra (es tracta com no-foreground en aquesta implementació).
5. `255` objecte.
6. `85/170/...` ignorades com no avaluables.

Calcula:

1. `TP`, `TN`, `FP`, `FN`.
2. `accuracy`, `precision`, `recall`, `f1`.

Important en defensa:

1. `accuracy` pot ser alta inclús amb detecció dolenta si hi ha molt fons.
2. Per això cal mirar també `precision/recall/F1`.

### 3.14 Informe de mètriques a disc

Referència: main.py:172

`escriu_informe_metricas` desa un resum textual de tots els casos en `results/metrics.txt`.

## 4) Orquestració del `main()` pas a pas

Referència: main.py:186

**Pas 2**

1. Carrega llistes de `splits`.
2. Carrega tensors `train_input`, `test_input`, `train_gt`, `test_gt`.
3. Mostra dimensions i tipus.

**Pas 3**

1. Calcula `mu`, `sigma`.
2. Desa `results/mu.png` i `results/sigma.png`.

**Pas 4**

1. Segmentació simple amb `thr = 30`.
2. Desa màscares a `results/step4_simple`.
3. Mostra percentatge mig de píxels foreground.

**Pas 5**

1. Fa 3 experiments:
2. `(alpha=1.5, beta=5.0)`.
3. `(alpha=2.5, beta=15.0)`.
4. `(alpha=3.5, beta=25.0)`.
5. Desa cada prova a subcarpeta.
6. Guarda resultats a diccionari per avaluació posterior.

**Pas 6**

1. Aplica morfologia sobre la prova central (`a2.5_b15.0`).
2. Desa màscares refinades.
3. Genera dos vídeos:
4. Pre-morfologia (`step5_gaussia.mp4`).
5. Post-morfologia (`step6_morfologia.mp4`).

**Pas 7**

1. Calcula mètriques de `pas4`, de cada prova `pas5`, i `pas6`.
2. Imprimeix mètriques per consola.
3. Desa informe `metrics.txt`.

## 5) Justificació dels resultats obtinguts

Dels resultats vistos en execucions:

1. `pas4_simple` té `recall` alt però `precision` moderada.
2. `pas5` amb `alpha/beta` alts puja `precision` però cau `recall`.
3. `pas6_morfologia` redueix soroll (més net), però també pot eliminar parts de cotxe i baixar `recall`.

Interpretació de compromís:

1. Si prioritzes detectar gairebé tots els cotxes, prefereixes més `recall`.
2. Si prioritzes poques alarmes falses, prefereixes més `precision`.
3. No hi ha un únic òptim universal: depèn de l'objectiu del sistema.

## 6) Preguntes típiques de docent i respostes model

### Bloc A: Dades i preprocés

1. Pregunta: Per què treballes en grisos i no en color? Resposta: El problema es resol bé amb intensitat i es redueix dimensionalitat, cost computacional i complexitat del model.

2. Pregunta: Com assegures que train i test no es barregen? Resposta: Les llistes de `splits` defineixen explícitament els intervals i el codi només construeix model amb `train_input`.

3. Pregunta: Com controles que totes les imatges tinguin mateixa mida? Resposta: A `load_stack` es verifica cada frame contra `primera_mida` i s'atura amb error si hi ha inconsistència.

### Bloc B: Model matemàtic

1. Pregunta: Què representa `mu` exactament? Resposta: Valor esperat temporal de cada píxel del fons sobre el conjunt d'entrenament.

2. Pregunta: Què representa `sigma`? Resposta: Variabilitat temporal del píxel; zones amb més moviment o soroll tenen `sigma` alta.

3. Pregunta: Diferència entre `thr` i `alpha/beta`? Resposta: `thr` és llindar global fix. `alpha/beta` defineix llindar adaptatiu per píxel (`alpha*sigma+beta`).

4. Pregunta: Per què fas càlcul en `float32`? Resposta: Per evitar wrapping de `uint8` en restes; és crític per a correcció numèrica.

### Bloc C: Postprocessat

1. Pregunta: Per què `opening` i després `closing`? Resposta: `opening` elimina falsos positius petits; `closing` reconnecta i omple buits en objectes detectats.

2. Pregunta: Com has triat mides de kernel? Resposta: Empíricament, buscant reduir soroll sense menjar massa contorn del vehicle; 3x3 i 5x5 funcionen bé per 240x320.

### Bloc D: Avaluació

1. Pregunta: Per què no et quedes només amb `accuracy`? Resposta: Amb molt fons, `accuracy` pot ser alta encara que el model perdi molts cotxes.

Cal complementar amb `precision/recall/F1`.

2. Pregunta: Com tractes ombres del groundtruth? Resposta: En aquesta implementació, valor `50` es considera classe no-foreground (junt amb background) per al còmput de TP/TN/FP/FN.
3. Pregunta: Què passa amb píxels `85` i `170` del CDnet? Resposta: S'ignoren amb màscara `valid`; no contribueixen a mètriques perquè són no avaluables.

**Bloc E: Enginyeria i robustesa**

1. Pregunta: Què passa si falla el codec de vídeo? Resposta: Ara no hi ha fallback, perquè la pràctica assumeix MP4 disponible. Si falla, es llença error explícit per detectar el problema ràpid.
2. Pregunta: Com garantiries reproduïbilitat? Resposta: Entrades fixes via `.txt`, paràmetres explícits al codi, i outputs amb estructura de carpetes consistent.

## 7) Problemes típics i com els resoldria

1. Error de rutes relatives. Solució: executar des de carpeta projecte o passar a rutes absolutes basades en script.
2. `cv2.imread` retorna `None`. Solució: comprovar path, permisos, extensió i integritat del fitxer.
3. Vídeo buit o corrupte. Solució: validar `writer.isOpened()`, codec, permisos d'escriptura, i mida `(w,h)` correcta.
4. Mètriques molt estranyes. Solució: revisar mapping d'etiquetes GT, valid mask i alineació temporal entre predicció i GT.
5. `recall` molt baix amb `alpha/beta` alts. Solució: baixar llindar adaptatiu o reforçar només postprocessat, no fer detector massa conservador.

## 8) Decisions de paràmetres i com defensar-les

**`thr = 30`**

1. Dona detecció sensible en model simple.
2. Resulta útil com baseline.

**Proves `alpha/beta`**

1. `1.5/5`: més permissiu, més deteccions.
2. `2.5/15`: equilibri.
3. `3.5/25`: molt estricte, menys soroll però menys recall.

**Morfologia `open=3`, `close=5`**

1. Escala adequada a resolució 240x320.
2. Si kernel massa gran, desapareixen parts del vehicle.

## 9) Complexitat computacional (explicació oral curta)

1. Càrrega: O(N$H$W).
2. Model (`mean/std`): O(N$H$W).
3. Segmentació per prova: O(N$H$W).
4. Morfologia: O(N$H$W*kˆ2) aproximadament, depèn del kernel.
5. Avaluació: O(N$H$W).

És un pipeline lineal en nombre de píxels i frames, escalable per seqüències moderades.

## 10) Millores possibles si el docent demana "què faries després?"

1. Cerca automàtica de paràmetres (`grid search`) optimitzant F1.
2. Model de fons adaptatiu en línia (actualització temporal de `mu/sigma`).
3. Incorporar ROI per limitar carretera i evitar cel/vores.
4. Treure ombres amb criteris de crominància (si es treballa en color).
5. Filtrat per components connectats (mínima àrea de vehicle).
6. Calcular mètriques per frame per detectar escenes conflictives.

## 11) Mini guió de defensa oral (2-3 minuts)

1. "El sistema llegeix les seqüències separades en train/test amb llistes fixes per garantir reproduïbilitat."
2. "Modelo el fons amb mitjana i desviació per píxel en flotant per evitar errors numèrics de `uint8`."
3. "Comparo dos detectors: llindar global i model gaussià adaptatiu amb `alpha/beta`."
4. "Refino amb morfologia per eliminar soroll i compactar objectes."
5. "Genero vídeos overlay per inspecció visual i avaluo amb GT usant accuracy, precision, recall i F1."
6. "Mostro compromís entre precisió i sensibilitat variant `alpha/beta`, i justifico escollir un punt de funcionament segons objectiu."

## 12) Preguntes de validació molt probables (respostes ultracurtes)

1. Per què no uses bucles per píxel? Per eficiència: NumPy vectoritzat és molt més ràpid i net.

2. Per què guardes màscares en 0/255 i no 0/1? Per compatibilitat amb visualització i eines d'imatge estàndard.
3. Per què es pot tenir accuracy alta i F1 baixa? Per desbalanceig fort: hi ha molt fons i pocs píxels de vehicle.
4. Què vol dir pujar `alpha`? Augmentar exigència local segons variabilitat, normalment baixa recall.
5. Què aporta la morfologia? Reduir soroll puntual i reparar forats del blob.

## 13) Autocrítica tècnica (molt útil en defensa)

1. El model de fons és estàtic; pot degradar-se si canvia llum globalment.
2. No hi ha component de tracking temporal d'objectes.
3. La morfologia és fixa; idealment s'hauria d'ajustar per escena.
4. L'avaluació tracta ombra com no-vehicle; segons criteri, es podria separar anàlisi.

Aquesta autocrítica suma punts perquè mostra criteri d'enginyeria.

## 14) Fitxers rellevants del projecte

1. Codi principal Python: main.py
2. Splits de dades: train_input.txt, test_input.txt, train_gt.txt, test_gt.txt
3. Informe mètriques Python: metrics.txt
4. Implementació alternativa MATLAB: matlab_main.m

## 15) Conclusió

La solució implementada és coherent amb els objectius del laboratori, cobreix tot el flux de processament i incorpora validació quantitativa. El codi està estructurat per ser defensable: cada pas té motivació, control d'errors, resultats guardats i mètriques interpretables.

En una validació oral, la clau és defensar:

1. Correcció numèrica (`float32` a diferències).
2. Rellevància dels paràmetres (`thr`, `alpha`, `beta`, morfologia).
3. Lectura crítica de mètriques (no quedar-se només amb accuracy).
4. Capacitat d'identificar limitacions i proposar millores realistes.

## 16) Diferències entre Python i MATLAB en aquesta pràctica

El punt clau és que **l'algorisme és el mateix** en tots dos entorns, però canvia la manera d'expressar-lo, les llibreries i alguns detalls de dades.

### 16.1 Què es manté igual

1. Mateix pipeline: càrrega, model de fons, segmentació, morfologia, avaluació.
2. Mateixes fórmules: `mu`, `sigma`, `|I-mu|>thr`, `|I-mu|>alpha*sigma+beta`.
3. Mateixa lògica d'interpretació de mètriques.

### 16.2 Diferències pràctiques importants

1. Forma dels tensors habitual:
2. Python/NumPy: sovint `N x H x W`.
3. MATLAB: sovint `H x W x N`.
4. Indexació:
5. Python comença a 0.
6. MATLAB comença a 1.
7. APIs d'imatge:
8. Python: `cv2.imread`, `cv2.imwrite`, `cv2.VideoWriter`, `cv2.morphologyEx`.
9. MATLAB: `imread`, `imwrite`, `VideoWriter`, `imopen/imclose`.
10. Conversió de tipus:
11. Python: `astype(np.float32)`.
12. MATLAB: `single(...)`.

### 16.3 Missatge defensable davant docent

No és que un entorn "sigui millor" en abstracte: **si vectoritzes bé**, els dos resolen el problema de manera equivalent. Python dona flexibilitat d'ecosistema, MATLAB dona molta comoditat de prototipat matricial. El criteri important és la coherència de dades, càlcul en flotant i validació quantitativa.

## 17) Tasca 1.4.8 - Com obtindria velocitats dels vehicles

Aquesta part no es programa aquí, però sí que es pot definir un sistema viable i defensable.

### 17.1 Proposta de sistema (alt nivell)

1. Detectar vehicles per frame (ja ho tenim amb segmentació).
2. Fer tracking multiobjecte (assignar un ID consistent a cada cotxe al llarg del temps).
3. Mesurar desplaçament en una zona calibrada.
4. Convertir de píxels a metres.
5. Calcular velocitat amb `v = distància / temps`.

### 17.2 Implementació conceptual pas a pas

1. **Detecció**
2. A partir de màscares binàries, extreure blobs i centroides.
3. **Tracking**
4. Relacionar deteccions entre frames per mantenir identitat del vehicle.
5. Opció robusta: Kalman + Hungarian matching.
6. **Zona de mesura**
7. Definir dues línies virtuals sobre la carretera: A i B.
8. Guardar `tA` quan el cotxe creua A i `tB` quan creua B.
9. **Calibració geomètrica**
10. Estimar relació píxel→metre (idealment amb homografia del pla carretera).
11. Amb això, obtenir distància real entre A i B.
12. **Velocitat**
13. `delta_t = tB - tA`.
14. `v_m_s = dist_real / delta_t`.
15. `v_km_h = v_m_s * 3.6`.

### 17.3 Punts conflictius i solucions

1. **Perspectiva**
2. Problema: mateix moviment en píxels no equival a mateixa distància real.
3. Solució: homografia (bird's-eye) o calibració per zones.
4. **Oclusions entre vehicles**
5. Problema: dos cotxes es tapen i es perd ID.
6. Solució: tracking predictiu (Kalman) i manteniment temporal de tracks.
7. **Soroll i ombres**
8. Problema: blobs trencats o falsos positius.
9. Solució: morfologia, filtres d'àrea, ROI i tractament específic d'ombres.
10. **FPS/temps real**
11. Problema: velocitat mal calculada si el temps no és fiable.
12. Solució: usar FPS real/metadades i timestamps consistents.

### 17.4 Idees “imaginatives” que aporten valor

1. Estimar velocitat amb optical flow dins del blob del vehicle.
2. Model espai-temps (line-scan) i obtenir velocitat per pendent de traces.
3. Fusió detecció clàssica + tracker robust per estabilitzar mesures.

### 17.5 Frase curta per defensa oral

“Per estimar velocitats, detectaria i traquejaria cada vehicle, mesuraria el temps de pas entre dues línies virtuals calibrades en metres i aplicaria `v=d/t`; els reptes principals són perspectiva, oclusions i soroll, resolts amb homografia, tracking predictiu i postprocessat morfològic.”